ROME Demo Assignment

Hierarchical LLM-assisted RTL generation with student-integrated coverage-guided testbench refinement

Course: LLM4ChipDesign | Instructors: Ramesh Karri, Weihua Xiao

Overview

Start from the **original ROME_Demo.ipynb**. First run the existing hierarchical RTL-generation and verification workflow. Then independently integrate the course **iVerilog + Covered** coverage-analysis pipeline into ROME. **No pre-integrated coverage version of the ROME notebook is provided.**

Required course resources

- Original ROME notebook: https://github.com/GUIDE-EDA/GUIDE/blob/main/colab-scripts/ROME_Demo.ipynb
- iVerilog + Covered reference flow: https://github.com/GUIDE-EDA/GUIDE/tree/main/colab-scripts/coverage_exp
- You are responsible for understanding both resources and combining them into one working verification workflow.

Learning goals

- Understand ROME's hierarchical RTL generation and iterative RTL-repair loop.
- Understand how coverage_exp uses iVerilog/vvp, waveform generation, Covered, and coverage reports.
- Integrate coverage analysis into an existing LLM-based RTL workflow rather than running a pre-built solution.
- Use uncovered RTL behavior to strengthen testbenches with additional targeted tests.
- Keep RTL refinement and testbench refinement as two clearly separated feedback loops.

Important design rule

Do not change RTL and testbench in the same iteration. During RTL repair, keep the testbench fixed. During testbench refinement, keep the RTL fixed. If a stronger coverage-guided testbench reveals a new RTL bug, freeze that stronger testbench and return to the ROME RTL-repair loop.

Conceptual workflow

- **ROME loop:** specification -> LLM-generated RTL -> iVerilog/vvp -> failure feedback -> repair RTL.
- **Coverage loop:** passing RTL + baseline TB -> Covered -> uncovered targets -> LLM adds targeted tests -> stronger TB -> rerun coverage.
- If the stronger TB causes functional failure, return to the ROME loop with the stronger TB held fixed.

Part I - Run the original ROME mux hierarchy

Run the original ROME notebook as provided. Generate the mux hierarchy and verify each level with the existing fixed self-checking testbenches.

- Generate **mux2_1, mux4_1, mux8_1, mux16_1, mux32_1, and mux64_1**.
- Use the existing ROME RTL feedback loop until each module passes iVerilog/vvp.
- Show the final successful simulation output for each module. The testbench must end with a clear **passed!** marker.
- Run the PPA optimizer and report the required PPA proxy results.
- You may also run Covered on mux64_1 to inspect the supplied testbench coverage, but no refinement is required if it is already close to exhaustive.

Part II - Extend ROME to a new hierarchical RTL design

Create a new hierarchical design with at least three modules, including a top module. The top module must instantiate at least two previously generated submodules.

- **Option A (recommended):** half_adder -> full_adder -> adder4 -> adder8.
- **Option B:** comparator hierarchy: cmp1 -> cmp4 -> cmp8, optionally with signed/unsigned variants.
- **Option C:** encoder/decoder hierarchy: pri_enc4 -> pri_enc8 -> top_wrapper with a valid flag.
- A design of similar complexity may be used with instructor approval.

Baseline testbenches

- Provide a self-checking baseline testbench for each module in your new hierarchy.
- The testbench must contain real functional checks and print **passed!** only when all checks succeed.
- Run the ROME RTL-repair loop with the testbench fixed until each generated RTL module passes.

Why the loops are separated

ROME repairs the RTL using compile/simulation feedback. The new coverage stage strengthens the testbench only after the RTL is functionally passing. Keeping the phases separate makes it clear whether an improvement comes from fixing the design or from changing the checker.

Part III - Integrate the iVerilog + Covered flow into ROME

Starting from the original ROME_Demo.ipynb, integrate the coverage-analysis workflow from coverage_exp into your verification pipeline. This integration work is part of the assignment.

Step 1 - Produce coverage data

- Modify or wrap the top-level self-checking testbench so that the simulation also produces the waveform data required by the coverage_exp flow.
- Use iVerilog/vvp to run the testbench.
- Use Covered to create the coverage database/report following the course coverage_exp example.
- Record line, toggle, and combinational-logic coverage.

Step 2 - Extract uncovered targets

- Inspect the detailed Covered report and identify uncovered lines, missing toggle directions, and uncovered logic combinations.
- Convert those misses into concise, actionable coverage feedback for the LLM.
- Preserve the reports for every coverage iteration.

Step 3 - Refine the testbench

- Provide the current testbench plus uncovered-target feedback to the LLM.
- **Do not ask the LLM to rewrite the entire testbench.** Ask it to add targeted test cases while preserving all existing checks.
- Rerun iVerilog/vvp and Covered after adding the new tests.
- Perform at least one refinement round beyond the baseline measurement. Additional rounds are encouraged if coverage is still improving.

Step 4 - Handle newly exposed RTL bugs

If a new coverage-guided test causes the RTL to fail, stop modifying the testbench. Keep the stronger testbench fixed, return to the ROME RTL-repair loop, and repair the RTL. Once the RTL passes again, resume coverage analysis.

Coverage history

Maintain **coverage_history.json** and include a compact summary table in your report.

Iteration	# tests	Line	Toggle 0->1	Toggle 1->0	Logic
Baseline	8	65%	55%	47%	42%
Refine 1	13	82%	71%	64%	68%
Refine 2	17	94%	84%	79%	83%

Part IV - Document the co-improvement process

- Show at least one ROME RTL-generation or RTL-repair iteration (compile error, warning, or failing functional check).
- Show the baseline coverage result after the RTL passes.
- Show at least one coverage-guided testbench refinement based on specific uncovered targets.
- If the refined TB exposes a new RTL failure, show the subsequent RTL repair with the TB held fixed.
- End with a final passing simulation and final coverage numbers.

Submission method

GitHub repository only (no email submission). Create or reuse your homework repository and place this assignment in a dedicated folder such as **HW_ROME_Coverage/**. Invite both **weihuax6@gmail.com** and **ajn313@nyu.edu** as collaborators.

What to submit

- **rome_submission.ipynb:** your completed notebook based on the original ROME_Demo.ipynb, including your own coverage integration code and visible outputs.
- **Generated RTL folders:** final .v files and ROME iteration logs for the mux demo and extension design.
- **Testbenches:** baseline TB plus coverage-refined versions (for example, tb_iter_1.v, tb_iter_2.v, tb_final.v).
- **Coverage artifacts:** Covered database/report files and coverage_history.json for each required run.
- **ROME_Report.pdf:** 2-3 pages summarizing hierarchy, RTL debugging, coverage integration, testbench refinement, and final results.

Report requirements

- State the LLM model(s) and important generation settings.
- Describe the hierarchy and module interfaces.
- Include the final prompt for the top module and one representative submodule.
- Explain how you integrated coverage_exp into the original ROME notebook.
- Include an iteration table that clearly separates RTL changes from TB changes.
- Include the coverage-history table and explain which uncovered targets motivated new tests.
- List final iVerilog/vvp commands and confirm that the final stronger TB passes.

Grading rubric

Criterion	Points
Original mux hierarchy: functional verification + PPA	20
New hierarchical design: meaningful decomposition + correct RTL	25
Self-checking baseline testbenches + ROME RTL loop	15
Student integration of coverage_exp + targeted TB refinement	25
RTL/TB iteration documentation, reproducibility, report quality	15

Academic integrity and responsible use

Using an LLM is required, but you must submit your own work. Do not share final prompts/code between teams. Treat all generated RTL and testbenches as potentially incorrect until verified. Never include API keys or credentials.